Definition

For real-valued nets f and g,

$\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M|g(x)|$ iff $f(x) \in O(g(x))$ and $|g(x)|$ is an asymptotic upper bound of $|f(x)|$.

$\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \ge M|g(x)|$ iff $f(x) \in \Omega(g(x))$ and $|g(x)|$ is an asymptotic lower bound of $|f(x)|$.

$\exists M_1, M_2 > 0, T \in \mathbb{R}: x \ge T \implies M_1|g(x)| \le |f(x)| \le M_2|g(x)|$ iff $f(x) \in \Theta(g(x))$.

$\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| < M|g(x)|$ iff $f(x) \in o(g(x))$.

$\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| > M|g(x)|$ iff $f(x) \in \omega(g(x))$.

$\in$ is also denoted as $=$ for them.

Limit

If $\exists k \in \mathbb{R}: x \ge k \implies g(x) \ne 0$,

$\limsup\left|\frac{f(x)}{g(x)}\right| < \infty$ iff $f(x) \in O(g(x))$.

Proof: $|f(x)| \le M|g(x)|, \quad x \ge T$; $\left|\frac{f(x)}{g(x)}\right| \le M, \quad x \ge T$; $\sup\left|\frac{f(x)}{g(x)}\right| \le M, \quad x \ge T$; $\limsup\left|\frac{f(x)}{g(x)}\right| \le M < \infty$;

$\liminf\left|\frac{f(x)}{g(x)}\right| > 0$ iff $f(x) \in \Omega(g(x))$.

Proof: $|f(x)| \ge M|g(x)|, \quad x \ge T$; $\left|\frac{f(x)}{g(x)}\right| \ge M, \quad x \ge T$; $\inf\left|\frac{f(x)}{g(x)}\right| \ge M, \quad x \ge T$; $\liminf\left|\frac{f(x)}{g(x)}\right| \ge M > 0$;

$\infty > \limsup\left|\frac{f(x)}{g(x)}\right| \ge \liminf\left|\frac{f(x)}{g(x)}\right| > 0$ iff $f(x) \in \Theta(g(x))$.

Proof: $|f(x)| \ge M_1|g(x)|, \quad x \ge T$; $\left|\frac{f(x)}{g(x)}\right| \ge M_1, \quad x \ge T$; $\inf\left|\frac{f(x)}{g(x)}\right| \ge M_1, \quad x \ge T$; $\liminf\left|\frac{f(x)}{g(x)}\right| \ge M_1 > 0$; $|f(x)| \le M_2|g(x)|, \quad x \ge T$; $\left|\frac{f(x)}{g(x)}\right| \le M_2, \quad x \ge T$; $\sup\left|\frac{f(x)}{g(x)}\right| \le M_2, \quad x \ge T$; $\limsup\left|\frac{f(x)}{g(x)}\right| \le M_2 < \infty$;

$\lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = 0$ iff $f(x) \in o(g(x))$. $|f(x)| < M|g(x)|$; $\left|\frac{f(x)}{g(x)}\right| < M$; $\lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = 0$;

$\lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = \infty$ iff $f(x) \in \omega(g(x))$. $|f(x)| > M|g(x)|$; $\left|\frac{f(x)}{g(x)}\right| > M$; $\lim_{x\to\infty}\left|\frac{f(x)}{g(x)}\right| = \infty$;

Relation

For any fixed $g(x)$, $o(g(x))$, $\Theta(g(x))$, and $\omega(g(x))$ are mutually exclusive.

Proof: Prove by contradiction. If $f \in o(g(x)) \cap \Theta(g(x))$, $\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| < M|g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R}: x \ge U \implies M_1|g(x)| \le |f(x)| \le M_2|g(x)|$; then set $M = M_1$, for $x \ge \max\{T, U\}$ $M_1|g(x)| \le |f(x)| < M_1|g(x)|$.; which can't happen. If $f \in \omega(g(x)) \cap \Theta(g(x))$, $\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| > M|g(x)|$; $\exists M_1, M_2 > 0, U \in \mathbb{R}: x \ge U \implies M_1|g(x)| \le |f(x)| \le M_2|g(x)|$; then set $M = M_2$, for $x \ge \max\{T, U\}$ $M_2|g(x)| \ge |f(x)| > M_2|g(x)|$.; which can't happen. If $f \in o(g(x)) \cap \omega(g(x))$, $\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| < M|g(x)|$; $\forall M > 0: \exists U \in \mathbb{R}: x \ge U \implies |f(x)| > M|g(x)|$; then set any fixed M, $M|g(x)| < |f(x)| < M|g(x)|$.; which can't happen.

For any fixed $g(x)$, $o(g(x)) \subseteq O(g(x))$;

Proof: Peak any $M > 0$.

For any fixed $g(x)$, $\omega(g(x)) \subseteq \Omega(g(x))$;

Proof: Peak any $M > 0$.

For any fixed $g(x)$, $\Theta(g(x)) = O(g(x)) \cap \Omega(g(x))$;

Proof: Prove $\Theta(g(x)) \subseteq O(g(x)) \cap \Omega(g(x))$; $\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M|g(x)|$; $\exists N > 0, U \in \mathbb{R}: x \ge U \implies |f(x)| \ge N|g(x)|$; then $\exists M, N > 0, \max\{T, U\} \in \mathbb{R}: x \ge \max\{T, U\} \implies N|g(x)| \le |f(x)| \le M|g(x)|$;

Prove $\Theta(g(x)) \supseteq O(g(x)) \cap \Omega(g(x))$; $\exists M_1, M_2 > 0, T \in \mathbb{R}: x \ge T \implies M_1|g(x)| \le |f(x)| \le M_2|g(x)|$; then $\exists M_2 > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M_2|g(x)|$; $\exists M_1 > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \ge M_1|g(x)|$;

For any fixed $g(x)$, $O(g(x)) \setminus (o(g(x)) \cup \Theta(g(x))) \ne \varnothing$;

Proof: Counterexample: $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}$; $g(x) = 1$; $f \in O(g(x))$: $\exists 1 > 0, 1 \in \mathbb{R}: x \ge 1 \implies |f(x)| \le |g(x)|$; $f \notin o(g(x))$: $\forall 1 > 0, T \in \mathbb{R}: \exists x \ge T, x \in \mathbb{Q}: |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$: $\forall M_1 > 0, x \in \mathbb{R}, x \notin \mathbb{Q}: M_1|g(x)| = M_1 > |f(x)| = 0$;

For any fixed $g(x)$, $\Omega(g(x)) \setminus (\omega(g(x)) \cup \Theta(g(x))) \ne \varnothing$;

Proof: Counterexample: $f(x) = 1$; $g(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ 0, & x \notin \mathbb{Q} \end{cases}$; $f \in \Omega(g(x))$: $\exists 1 > 0, 1 \in \mathbb{R}: x \ge 1 \implies |f(x)| \ge |g(x)|$; $f \notin \omega(g(x))$: $\forall 1 > 0, T \in \mathbb{R}: \exists x \ge T, x \in \mathbb{Q}: |f(x)| = |g(x)|$; $f \notin \Theta(g(x))$: $\forall M_2 > 0, x \in \mathbb{R}, x \notin \mathbb{Q}: M_2|g(x)| = 0 < |f(x)| = 1$;

$f(x) \in O(g(x)) \iff g(x) \in \Omega(f(x))$;

Proof: $\implies$: $\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R}: x \ge T \implies |g(x)| \ge \frac{1}{M}|g(x)|$; $\impliedby$: $\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \ge M|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R}: x \ge T \implies |g(x)| \le \frac{1}{M}|g(x)|$;

$f(x) \in o(g(x)) \iff g(x) \in \omega(f(x))$;

Proof: $\implies$: $\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| < M|g(x)|$; then $\forall \frac{1}{M} > 0: \exists T \in \mathbb{R}: x \ge T \implies |g(x)| > \frac{1}{M}|f(x)|$; $\impliedby$: $\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| > M|g(x)|$; then $\forall \frac{1}{M} > 0: \exists T \in \mathbb{R}: x \ge T \implies |g(x)| < \frac{1}{M}|f(x)|$;

For any fixed $g(x)$, $\exists f(x): f(x) \notin O(g(x)) \cup \Omega(g(x))$;

Proof: $f(x) = \begin{cases} 1, & x \in \mathbb{Q} \\ x, & x \notin \mathbb{Q} \end{cases}$; $g(x) = \begin{cases} x, & x \in \mathbb{Q} \\ 1, & x \notin \mathbb{Q} \end{cases}$; For $x \in \mathbb{R} \setminus \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M|g(x)|$; For $x \in \mathbb{Q}$, $\nexists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \ge M|g(x)|$;

Binary relation

$\Theta(g(x))$ is an equivalence class.

Proof: Transitive: $\exists M_1, M_2 > 0, T \in \mathbb{R}: x \ge T \implies M_1|g(x)| \le |f(x)| \le M_2|g(x)|$; $\exists N_1, N_2 > 0, U \in \mathbb{R}: x \ge U \implies N_1|h(x)| \le |g(x)| \le N_2|h(x)|$; then $\exists M_1N_1, M_2N_2 > 0, \max\{T, U\} \in \mathbb{R}: x \ge \max\{T, U\} \implies M_1N_1|h(x)| \le |f(x)| \le M_2N_2|h(x)|$; Reflexive: $f(x) \in \Theta(f(x))$; $\exists 1, 1 > 0, a \in \mathbb{R}: x \in \mathbb{R} \implies 1|f(x)| \le |f(x)| \le 1|f(x)|$; Symmetric: $\exists M_1, M_2 > 0, T \in \mathbb{R}: x \ge T \implies M_1|g(x)| \le |f(x)| \le M_2|g(x)|$; then $\exists \frac{1}{M_2}, \frac{1}{M_1} > 0, T \in \mathbb{R}: x \ge T \implies \frac{1}{M_2}|f(x)| \le |f(x)| \le \frac{1}{M_1}|f(x)|$;

$O(g(x))$ is partial-ordered set with Θ as $=$.

Proof: Transitive: $\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M|g(x)|$; $\exists N > 0, U \in \mathbb{R}: x \ge U \implies |g(x)| \le N|h(x)|$; then $\exists MN > 0, \max\{T, U\} \in \mathbb{R}: x \ge \max\{T, U\} \implies |f(x)| \le MN|h(x)|$; Reflexive: $\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M|g(x)|$; then $\exists \frac{1}{M} > 0, T \in \mathbb{R}: x \ge T \implies |g(x)| \le \frac{1}{M}|f(x)|$; Antisymmetric: $\exists M > 0, T \in \mathbb{R}: x \ge T \implies |f(x)| \le M|g(x)|$; $\exists N > 0, U \in \mathbb{R}: x \ge U \implies |g(x)| \le N|f(x)|$; then $\exists M, \frac{1}{N} > 0, \max\{T, U\} \in \mathbb{R}: x \ge \max\{T, U\} \implies \frac{1}{N}|g(x)| \le |f(x)| \le M|g(x)|$;

$\Omega(g(x))$ is partial-ordered set with Θ as $=$.

Proof: Similar to that of O.

$o(g(x))$ is a strictly partially-ordered set.

Proof: Asymmetry: $f(x) \in o(g(x)) \iff g(x) \in \omega(f(x)) \implies g \notin o(f(x))$; Transitivity: $\forall M > 0: \exists T \in \mathbb{R}: x \ge T \implies |f(x)| < M|g(x)|$; $\exists U \in \mathbb{R}: x \ge U \implies |g(x)| < |h(x)|$; then $\forall M > 0: \exists \max\{T, U\} \in \mathbb{R}: x \ge \max\{T, U\} \implies |f(x)| < M|h(x)|$;

$\omega(g(x))$ is a strictly partially-ordered set.

Proof: Similar to that of o.

Integral test (for convergence) or Maclaurin – Cauchy test

Suppose $f(x): [a, \infty) \to \mathbb{R}$ for some $a \in \mathbb{N}$ is monotone and $\lim_{x\to\infty} f(x) = 0$. Then $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent iff $\sum_{k=a}^{\infty} f(k)$ is convergent. And,

If $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent and $f(x)$ is monotone decreasing, for any $n \in \mathbb{N}_{\ge a}$, $\int_n^\infty f(x)\,\mathrm{d}x \le \sum_{k=n}^{\infty} f(k) \le f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{>a}$, $f(n) + \int_n^\infty f(x)\,\mathrm{d}x \le \int_{n-1}^\infty f(x)\,\mathrm{d}x$.

If $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent and $f(x)$ is monotone increasing, for any $n \in \mathbb{N}_{\ge a}$, $\int_n^\infty f(x)\,\mathrm{d}x \ge \sum_{k=n}^{\infty} f(k) \ge f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; called remainder estimate for integral test. Note that for $n \in \mathbb{N}_{>a}$, $f(n) + \int_n^\infty f(x)\,\mathrm{d}x \ge \int_{n-1}^\infty f(x)\,\mathrm{d}x$.

Proof: Consider the case where $f(x)$ is monotone increasing. The case where $f(x)$ is monotone decreasing can be proven similarly. First consider the case where $\int_a^\infty f(x)\,\mathrm{d}x$ is convergent. This implies that $f(x)$ is non-positive. Thus, for any fixed $n \in \mathbb{N}_{\ge a}$, $\int_n^{n+1} f(x)\,\mathrm{d}x \ge f(n)$,; and for any $k \ge n + 1$, $\int_k^{k+1} f(x)\,\mathrm{d}x \ge f(k) \ge \int_{k-1}^{k} f(x)\,\mathrm{d}x$.; For any $N \in \mathbb{N}_{\ge n}$, summing over all $k \in \mathbb{N}_{\ge n}$, we obtain, $\int_n^{N+1} f(x)\,\mathrm{d}x \ge \sum_{k=n}^{N} f(k) \ge f(n) + \int_n^N f(x)\,\mathrm{d}x$.; Take limit $\lim_{N\to\infty}$, we obtain $\int_n^\infty f(x)\,\mathrm{d}x \ge \sum_{k=n}^{\infty} f(n) \ge f(n) + \int_n^\infty f(x)\,\mathrm{d}x$.; Second, consider the case where $\int_a^\infty f(x)\,\mathrm{d}x$ is divergent. Since $f(x)$ is monotone. Either $\int_a^\infty f(x)\,\mathrm{d}x = -\infty$ or $\int_a^\infty f(x)\,\mathrm{d}x = \infty$. For the former, the supremum of $f(x)$ is non-positive, and $\sum_{k=a}^{\infty} f(k) = -\infty$. For the latter, there exists $x_0 > a$ such that $f(x) > 0$ for all $x \ge x_0$, and thus $\sum_{k=a}^{\infty} f(k) = \infty$.

p-test (for improper integral)

For any fixed $a > 0$, $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x$ converges iff $p > 1$, and if so, $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x = \frac{1}{(p-1)\,a^{p-1}}$.; $\int_0^a \frac{1}{x^p}\,\mathrm{d}x$ converges iff $p < 1$, and if so, $\int_0^a \frac{1}{x^p}\,\mathrm{d}x = -\frac{1}{(p-1)\,a^{p-1}}$.; Proof: $\int \frac{1}{x^p}\,\mathrm{d}x = \begin{cases} -\frac{1}{(p-1)\,x^{p-1}}, & p \ne 1 \\ \ln(x), & p = 1 \end{cases}$; $\forall p < 1: \nexists \lim_{x\to\infty} -\frac{1}{(p-1)\,x^{p-1}}$; $\nexists \lim_{x\to\infty} \ln(x)$; $\forall p > 1: \lim_{x\to\infty} -\frac{1}{(p-1)\,x^{p-1}} = 0$ $\int_a^\infty \frac{1}{x^p}\,\mathrm{d}x = \frac{1}{(p-1)\,a^{p-1}}$; $\forall p > 1: \nexists \lim_{x\to 0+} -\frac{1}{(p-1)\,x^{p-1}}$; $\nexists \lim_{x\to 0+} \ln(x)$; $\forall p < 1: \lim_{x\to 0+} -\frac{1}{(p-1)\,x^{p-1}} = 0$ $\int_0^a \frac{1}{x^p}\,\mathrm{d}x = -\frac{1}{(p-1)\,a^{p-1}}$;

Generalized p-test (for improper integral)

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, \quad x \in \mathbb{R}_{>0}$; $L_n(x) = \ln(L_{n-1}(x)), \quad n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.; For any fixed $a > 1, k \in \mathbb{N}$, $\int_a^\infty \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right)(L_k(x))^m}\,\mathrm{d}x$; converges iff $m > 1$.

Proof: Let $u = L_k(x)$; $\mathrm{d}u = \frac{\mathrm{d}x}{\prod_{i=0}^{k-1} L_i(x)}$; $\int \frac{1}{\left(\prod_{i=0}^{k-1} L_i(x)\right)(L_k(x))^m}\,\mathrm{d}x = \int \frac{1}{u^m}\,\mathrm{d}u$; By p-test, the statement holds.

For any fixed $0 < a < 1$, $\int_0^a \frac{1}{x(\ln x)^m}\,\mathrm{d}x$; converges iff $m < 1$.

Proof: By substitution of $u = \ln x$.

Generalized p-test for series or generalized p-series test

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, \quad x \in \mathbb{R}_{>0}$; $L_n(x) = \ln(L_{n-1}(x)), \quad n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.; For any fixed $a \in \mathbb{N}$, $\sum_{n=a}^{\infty} \frac{1}{\left(\prod_{i=0}^{k-1} L_i(n)\right)(L_k(n))^m}$; converges iff $m > 1$.

Proof: By integral test and generalized p-test for improper integral, the statement holds.

Generalized Master Theorem

For a positive valued net $T(n)$ defined on $n = b^h$ for any $h \in \mathbb{N}_0$ as $T(n) = aT\left(\frac{n}{b}\right) + f(n)$; for some constants $a > 0$ and $b > 1$, and non-negative-valued function $f(n)$,

If for some $c < \log_b a$, $f(n) = O(n^c)$,; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

Define iterated logarithmic function $L_n(x)$ as $L_0(x) = x, \quad x \in \mathbb{R}_{>0}$; $L_n(x) = \ln(L_{n-1}(x)), \quad n \in \mathbb{N}, x \in \mathbb{R}_{>0}$.;

If for some positive integers k and positive number m, $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}(L_i(n))^{-1}\right)(L_k(n))^{m-1}\right)$,; then $T(n) = \Theta\left(n^{\log_b a}(L_k(n))^m\right)$.;

If for some positive integer k, $f(n) = \Theta\left(n^{\log_b a}\prod_{i=1}^{k}(L_i(n))^{-1}\right)$,; then $T(n) = \Theta\left(n^{\log_b a}L_{k-1}(n)\right)$.;

If for some positive integers k and some positive number m, $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}(L_i(n))^{-1}\right)(L_k(n))^{-m-1}\right)$,; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

If $f(n) = \Omega(n^c)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, k < 1$ such that $n \ge m$ implies $af\left(\frac{n}{b}\right) \le kf(n)$; is satisfied, then $T(n) = \Theta(f(n))$.;

Proof: Let $s = \log_b a$, $n = b^h$, and $f(x) = x^s g(x)$. At level j, there are a^j nodes, each of size $\frac{n}{b^j}$. Thus, the total non-recursive work at level j is $a^j f\left(\frac{n}{b^j}\right)$.; Hence $T(n) = \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1) = \sum_{j=0}^{h-1} b^{sj} f\left(\frac{b^h}{b^j}\right) + \Theta(n^s) = n^s \sum_{j=0}^{h-1} g\left(b^{h-j}\right) + \Theta(n^s) = n^s \sum_{r=1}^{h} g(b^r) + \Theta(n^s)$; Case 1: If for some $c < \log_b a$, $f(n) = O(n^c)$,; then $T(n) = \Theta\left(n^{\log_b a}\right)$.; Case 2: $L_k(b^x) = \Theta(L_{k-1}(x))$; $f(n) = \Theta\left(n^{\log_b a}\left(\prod_{i=1}^{k-1}(L_i(n))^{-1}\right)(L_k(n))^{m-1}\right)$; $g(b^r) = \Theta\left(\left(\prod_{i=1}^{k-2}(L_i(r))^{-1}\right)(L_{k-1}(r))^{m-1}\right)$; By generalized p-test for series: Sub-case 1 $m > 0$: $T(n) = \Theta\left(n^{\log_b a}(L_k(n))^m\right)$.; Sub-case 2 $m = 0$: $T(n) = \Theta\left(n^{\log_b a}L_{k-1}(n)\right)$.; Sub-case 3 $m = 0$: $T(n) = \Theta\left(n^{\log_b a}\right)$.; Case 3: If $f(n) = \Omega(n^c)$ for some $c > \log_b a$, and the regularity condition that $\exists m \in \mathbb{R}, 0 < k < 1$ such that $n \ge m$ implies $af\left(\frac{n}{b}\right) \le kf(n)$; is satisfied, then $af\left(\frac{n}{b}\right) \le kf(n)$; $a^j f\left(\frac{n}{b^j}\right) \le a^{j-1}kf\left(\frac{n}{b^{j-1}}\right) \le k^j f(n)$; $T(n) = \sum_{j=0}^{h-1} a^j f\left(\frac{n}{b^j}\right) + n^s T(1) = f(n) + \sum_{j=1}^{h-1} O\left(k^j f(n)\right) + \Theta(n^s)$; $\sum_{j=1}^{\infty} k^j f(n) = \frac{f(n)}{1-k}$; $T(n) = f(n) + O(f(n)) + \Theta(n^s) = \Theta(f(n))$;

Terms:

h is called recursion level.

$\log_b a$ is called critical exponent c_{crit}.

$T(1)$ is called leaf cost.

At each node, the first term is called recursive work, and the second term is called non-recursive work.

For original master theorem, the second case is

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}(\ln n)^{m-1}\right)$,; then $T(n) = \Theta\left(n^{\log_b a}(\ln n)^m\right)$.;

If $f(n) = o\left(n^{\log_b a}(\ln n)^{-1}\right)$,; then $T(n) = \Theta\left(n^{\log_b a}\ln(\ln n)\right)$.;

If for some positive integers m, $f(n) = \Theta\left(n^{\log_b a}(\ln n)^{-m-1}\right)$,; then $T(n) = \Theta\left(n^{\log_b a}\right)$.;

Stirling's approximation

$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n\left(1 + O\left(\frac{1}{n}\right)\right)$;

特徵方程式

定義：對一個給定 k 階常係數線性遞迴數列：$< a_n >$: $a_n = \sum_{i=1}^{k} c_i \cdot a_{n-i} + f(n)$; ，其特徵方程式為 $x^k - \sum_{i=1}^{k} c_i \cdot x^{k-i} = 0$，特證方程式的根稱其特徵根。

若一個給定 k 階常係數線性齊次遞迴數列 $< a_n >$ 有 a 相異特徵根 $q_1, q_2, q_3, \ldots, q_a$，其中第 i 個根 $(1 \le i \le a)$ 為 m_i 重根，則 $< a_n >$ 之一般式為：$a_n = \sum_{i=1}^{a}\left(\sum_{j=1}^{m_i}\left(A_{i,j} \cdot n^{j-1}\right) \cdot q_i{}^n\right)$;

若給定一個 k 階常係數線性非齊次遞迴數列：$< a_n >$: $a_n = \sum_{i=1}^{k} c_i \cdot a_{n-i} + f(n)$; 則令：$a_n = a'_n + a''_n$; 使得 $< a'_n >$ 為 k 階常係數線性齊次遞迴數列，$< a''_n >$ 為 k 階常係數線性非齊次遞迴數列且一次項同 $< a'_n >$ 且 $f(n)$ 項同 $< a_n >$。$a''_n = g(n)$, $\quad \deg(g(n)) \le \deg(f(n))$; 解出 k 階常係數線性非齊次遞迴數列 $< a'_n >$ 與 $g(n)$ ，即可解出 $< a_n >$。

Linear Data Structures

An unordered list is a list where each element holds a relative position with respect to the others. Each element is called node. If applicable, the first node is called head and the last node is called tail. An ordered list is a list which are ordered based upon some underlying characteristic of the elements. A stack is a last-in-first-out (LIFO) list. Adding an element is called push and is O(1), the last pushed element is called top, and the first pushed element is called bottom. Removing and returning top is called pop and is O(1). Peeking top is called top and is O(1). A queue is first-in-first-out (FIFO) list. Adding an element is called enqueue and is O(1), the first enqueued element is called front, and the last enqueued element is called back. Removing and returning front is called dequeue and is O(1). Peeking front is called peek and is O(1). A doubly-ended queue (deque) is a list that is simultaneously a stack and a queue, that is, has O(1) peeking, pushing, and popping from both ends. A dynamic array is a contiguous array that reserves a new capacity elsewhere and moves to it whenever its size reaches its capacity. It has O(1) random access and popping back, amortized O(1) pushing back, but O(n) removal and insertion. It is a stack with front as bottom and back as top. A singly linked list is an unordered list where each node stores only the next node. It has O(1) peeking, pushing, and popping front and insertion after node, but O(n) random access, removal of node, and insertion before node. If the tail is stored, it has O(1) peeking and pushing back. It is a stack with head as top and tail as bottom. If the tail is stored, it is a queue with the tail as front and head as back. A doubly linked list is an unordered list where each node stores the previous node and the next node. It has O(1) peeking, pushing, and popping front and back, insertion after and before node, and removal of node, but O(n) random access. It is a deque. An implementation of deque is a dynamic array of fixed-size blocks, which has O(1) random access and popping front and back, amortized O(1) pushing front and back, but O(n) removal and insertion. To keep track of the minimum element of a stack in O(1) time with O(n) extra space. One can maintain the original stack of elements and an extra stack of extrema. When an element is pushed to the original stack, it is compared with the top of the extrema stack and the minimum one is pushed to the extrema stack (for the first element, it is pushed to the extrema stack). When an element is popped from the original stack, an element is popped from the extrema stack. This way, the top of the extrema stack is always the minimum of the elements in the original stack.

```
# Python operator precedence order
# ()
# ** (a ** b is a to the power of b, also pow
        (a, b))
# +x -x ~x (bitwise NOT)
# * / (float quotient) // (floor quotient) %
        (modulo)
# + - > << >> (preserve sign)
# & (bitwise AND)
# \^ (bitwise XOR)
# | (bitwise OR)
# == != > >= < <= (value comparison) is is
        not (identity) in not in
# not (logical NOT)
# and (logical AND)
# or (logical OR)
bool(0)  # False, only 0 False, other numbers
         True
2**72  # int is of infinite precision, ** is
        power
float("inf")  # infinite float
(1).__add__(2)  # 3, must use (1). since 1.
        is treated as decimal
3 // 2  # 1
-3 // 2  # -2
3 / 2  # 1.5
type(3 / 3)  # float
divmod(3, 2)  # (1, 1), the pair (x // y, x %
         y)
abs(-2)  # 2
c = complex(1, 2)  # (1+2j), complex(re, im)
c.conjugate()  # (1-2j)
a = 0.5  # a == 0.5
type(True)  # <class 'bool'>
type(1)  # <class 'int'>
type(1.0)  # <class 'float'>
type(complex(1, 1))  # <class 'complex'>
a = None
a is None  # True
a = [1]
b = a
a == b  # True
a != b  # False
a is b  # True, i.e., id(a) == id(b)
a is not b  # False
b = a.copy()  # Shallow copy
a == b  # True
a is b  # False
a[0] is b[0]  # True
c, d = a, b
c, d = d, c  # swap
3 if 2 > 1 else 2  # 3
sradius = input("Please enter the radius of
        the circle ")
# ...
# Please enter the radius of the circle (
        input here, say 1)
# sradius == 1
# print(obj[, obj ,...], sep=' ', end='\n',
        file=sys.stdout, flush=False)
# obj: Arbitrary number of objects that can
        be converted to str. Will be
        converted to str and printed.
# sep: Separator between objects
# end: What to print at end
# file: An object with a write method to
        print to
# flush: True for flushed, False for buffered
print("a")  # a
print(1, [1])  # 1 [1]
print(1, 2, sep=",", end="1")  # 1,21<
        no_newline>
print(1, 2, sep=", ", end="")  # 1, 2<
        no_newline>


a = "a"
type(a)  # <class 'str'>
a += "b"  # a == 'ab'
"He".__add__("llo")  # 'Hello'
str(1)  # '1'
"g" in "egg"  # True
"f" in "egg"  # False
len("Hello")  # 5
"Hello World".capitalize()  # 'Hello world',
        first capitalize, other lowercase
# str.center(width, fillchar=' '), Return
        centered in a string of length width.
         Padding is done using the specified
        fillchar (default is an ASCII space).
         The original string is returned if
        width is less than or equal to len(s
        ).
"Python".center(10)  # '  Python  '
"Python".center(10, "-")  # '--Python--'
"Python".center(4)  # 'Python'
# str.count(sub[, start[, end]])
"spam, spam, spam".count("spam")  # 3
"spam, spam, spam".count("spam", 5)  # 2
"spam, spam, spam".count("spam", 5, 10)  # 1
"spam, spam, spam".count("eggs")  # 0
"spam, spam, spam".count("")  # 17
# str.endswith(suffix[, start[, end]]),
        Return True if the string ends with
        the specified suffix, otherwise
        return False. suffix can also be a
        tuple of suffixes to look for. With
        optional start, test beginning at
        that position. With optional end,
        stop comparing at that position.
        Using start and end is equivalent to
        str[start:end].endswith(suffix).
# suffix can also be a tuple of suffixes to
        look for.
"Python".endswith("on")  # True
"a tuple of suffixes".endswith(("at", "in"))
         # False
"a tuple of suffixes".endswith(("at", "es"))
         # True
"Python is amazing".endswith("is", 0, 9)  #
        True
# str.startswith(prefix[, start[, end]]),
        endswith but startswith
# str.expandtabs(tabsize=8)
"01\t012\t0123\t01234".expandtabs()  # '01
            012     0123    01234'
"01\t012\t0123\t01234".expandtabs(4)  # '01
        012 0123    01234'
# str.find(sub[, start[, end]]), Return the
        lowest index in the string where
        substring sub is found within the
        slice s[start:end]. Optional
        arguments start and end are
        interpreted as in slice notation.
        Return -1 if sub is not found.
"spam, spam, spam".find("sp")  # 0
"spam, spam, spam".find("sp", 5)  # 6
# str.rfind(sub[, start[, end]]), find but
        highest index
# str.index(sub[, start[, end]]), Like find
        (), but raise ValueError when the
        substring is not found.
"spam, spam, spam".index("spam")  # 0
# 'spam, spam, spam'.index('eggs')  #
        ValueError: substring not found
```

```
# str.rindex(sub[, start[, end]]), index but
        highest index
# str.join(iterable), Return a string which
        is the concatenation of the strings
        in iterable. A TypeError will be
        raised if there are any non-string
        values in iterable, including bytes
        objects. The separator between
        elements is the string providing this
         method.
”, ”.join([”spam”, ”spam”, ”spam”])  # ’spam,
         spam, spam’
”-”.join(”Python”)  # ’P-y-t-h-o-n’
””.join([”a”, ”b”, ”c”])  # ’abc’
”Hello World”.lower()  # ’hello world’, all
        lowercase
# str.lstrip(chars=None), Return a copy of
        the string with leading characters
        removed. The chars argument is a
        string specifying the set of
        characters to be removed. If omitted
        or None, the chars argument defaults
        to removing whitespace. The chars
        argument is not a prefix; rather, all
         combinations of its values are
        stripped.
”   spacious   ”.lstrip()  # ’spacious   ’
”www.example.com”.lstrip(”cmowz.”)  # ’
        example.com’
# str.rstrip(chars=None), lstrip but trailing
# str.strip(chars=None), lstrip and rstrip
# str.partition(sep), Split the string at the
         first occurrence of sep, and return
        a 3-tuple containing the part before
        the separator, the separator itself,
        and the part after the separator. If
        the separator is not found, return a
        3-tuple containing the string itself,
         followed by two empty strings.
”Monty Python”.partition(” ”)  # (’Monty’, ’
        ’, ’Python’)
”Monty Python’s Flying Circus”.partition(” ”)
         # (’Monty’, ’ ’, ”Python’s Flying
        Circus”)
”Monty Python”.partition(”-”)  # (’Monty
        Python’, ’’, ’’)
# str.rpartition(sep), partition but last
        occurrence
# str.removeprefix(prefix), If the string
        starts with the prefix string, return
         string[len(prefix):]. Otherwise,
        return a copy of the original string:
”Arthur: three!”.removeprefix(”Arthur: ”)  #
        ’three!’
# str.removesuffix(suffix, /), If the string
        ends with the suffix string and that
        suffix is not empty, return string[:-
        len(suffix)]. Otherwise, return a
        copy of the original string
# str.replace(old, new, count=-1), Return a
        copy of the string with all
        occurrences of substring old replaced
         by new. If count is given, only the
        first count occurrences are replaced.
         If count is not specified or -1,
        then all occurrences are replaced.
”spam, spam, spam”.replace(”spam”, ”eggs”)  #
         ’eggs, eggs, eggs’
”spam, spam, spam”.replace(”spam”, ”eggs”, 1)
          # ’eggs, spam, spam’
# str.split(sep=None, maxsplit=-1), Return a
        list of the words in the string,
        using sep as the delimiter string. If
         maxsplit is given, at most maxsplit
        splits are done, the leftmost ones (
        thus, the list will have at most
        maxsplit+1 elements). If maxsplit is
        not specified or -1, then there is no
         limit on the number of splits (all
        possible splits are made). If sep is
        given, consecutive delimiters are not
         grouped together and are deemed to
        delimit empty strings (for example,
        ’1,,2’.split(’,’) returns [’1’, ’’,
        ’2’]). The sep argument may consist
        of multiple characters as a single
        delimiter. Splitting an empty string
        with a specified separator returns
        [’’]. If sep is not specified or is
        None, a different splitting algorithm
         is applied: runs of consecutive
        whitespace are regarded as a single
        separator, and the result will
        contain no empty strings at the start
         or end if the string has leading or
        trailing whitespace. Consequently,
        splitting an empty string or a string
         consisting of just whitespace with a
         None separator returns [].
”1,2,3”.split(”,”)  # [’1’, ’2’, ’3’]
”1,2,3”.split(”,”, maxsplit=1)  # [’1’,
        ’2,3’]
”1,2,,3,”.split(”,”)  # [’1’, ’2’, ’’, ’3’,
        ’’]
”1<>2<>3<4”.split(”<>”)  # [’1’, ’2’, ’3<4’]
# str.rsplit(sep=None, maxsplit=-1), split
        but rightmost
# str.swapcase()
”Hello World”.swapcase()  # ’hELLO wORLD’
# str.title(), Return a titlecased version of
         the string where words start with an
         uppercase character and the
        remaining characters are lowercase.
        The algorithm uses a simple language-
        independent definition of a word as
        groups of consecutive letters.
”Hello world”.title()  # ’Hello World’
”they’re bill’s friends from the UK”.title()
         # ”They’Re Bill’S Friends From The
        Uk”
# str.upper(), all uppercase

# A list is a dynamic array of elements of
        arbitrary types
a = [10, 20, 30]
type(a)  # <class ’list’>
a[0]  # 10, O(1)
a[1]  # 20, O(1)
a[-1]  # 30, O(1)
a[-2]  # 20, O(1)
len(a)  # 3, O(1)
a.__len__()  # 3, O(1)
a.append(40)  # a == [10, 20, 30, 40],
        amortized O(1)
a.pop()  # a == [10, 20, 30], O(1)
a.insert(1, 50)  # a == [10, 50, 20, 30], O(n
        )
a.pop(0)  # a == [50, 20, 30], O(n)
a[0] = 0  # a == [0, 20, 30], O(1)
b, c, d = a  # b == 0, c == 20, d == 30
a.clear()  # a == []
a = [0, 1, 2]
b = a  # id(a) == id(b)
b.pop()  # a == b == [0, 1], id(a) == id(b)
b = [1, 2]  # id(a) != id(b), a == [0, 1]
a = [1, 1, 1]
a.count(1)  # 3, O(n)
b = a.copy()  # a == b but id(a) != id(b)
a.index(1)  # 0, first occurrence, O(n)
a.extend([0, 4, 3, 4]),  # a == [1, 1, 1, 0,
        4, 3, 4], O(k)
a.remove(4)  # a.pop(a.index(x))
a.reverse()  # a == [4, 3, 0, 1, 1, 1], O(n)
a.sort()  # [0, 1, 1, 1, 3, 4], O(n log n)
        worst-case O(n) best-case, Powersort
sorted(a)  # same as a.sort()
del a[0:5:2]  # a == [1, 1, 4]
1 in a  # True
0 not in a  # True
a = [None] * 5  # a == [None, None, None,
        None, None]
del a  # a is now not defined
a = [1, ”1”, [1], True]  # arbitrary types
a.__getitem__(0)  # 1, same as a[0]
a.__setitem__(0, (1,))  # a == [(1,), ’1’,
        [1], True], same as a[0] = (1,)
it = iter([1, 2, 3])  # also it = [1, 2, 3].
        __iter__()
iter(a) is iter(a)  # False
next(it)  # 1, also it.__next__()
# Return of __next__ is defined by the yield
        clause in the method.
# __next__ is used by for loop.
# range(start, stop, step), all are integers,
         start + k step, k = [0, (stop -
        start) / step - 1] integers
reversed(a)  # a reversed iterator of a, also
         a.__reversed__()
next(a)  # True
list(range(0, 10, 1))  # [0, 1, 2, 3, 4, 5,
        6, 7, 8, 9]
list(range(9, -1, -2))  # [9, 7, 5, 3, 1]
# List comprehensions provide a concise way
        to create lists. Common applications
        are to make new lists where each
        element is the result of some
        operations applied to each member of
        another sequence or iterable, or to
        create a subsequence of those
        elements that satisfy a certain
        condition.
list(map(lambda x: x**2, range(10)))  # [0,
        1, 4, 9, 16, 25, 36, 49, 64, 81]
[x**2 for x in range(10)]  # [0, 1, 4, 9, 16,
         25, 36, 49, 64, 81]
[(x, y) for x in [1, 2, 3] for y in [3, 1, 4]
         if x != y]  # [(1, 3), (1, 4), (2,
        3), (2, 1), (2, 4), (3, 1), (3, 4)]
[w.strip() for w in [”  banana”, ”
        loganberry ”, ”passion fruit  ”]]  #
        [’banana’, ’loganberry’, ’passion
        fruit’]
[num for elem in [[1, 2, 3], [4, 5, 6], [7,
        8, 9]] for num in elem]  # [1, 2, 3,
        4, 5, 6, 7, 8, 9]
[
    [
        row[i]
        for row in [
            [1, 2, 3, 4],
            [5, 6, 7, 8],
            [9, 10, 11, 12],
        ]
    ]
    for i in range(4)
]  # [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4,
        8, 12]]
# zip(*iterables, strict=False), Iterate over
         several iterables in parallel,
        producing tuples with an item from
        each one, similar to transposing a
        matrix. By default, zip() stops when
        the shortest iterable is exhausted.
        With strict=True, it raises a
        ValueError if one iterable is
        exhausted before the others.
list(
    zip(
        *[
            [1, 2, 3, 4],
            [5, 6, 7, 8],
            [9, 10, 11, 12],
        ]
    )
)  # [(1, 5, 9), (2, 6, 10), (3, 7, 11), (4,
        8, 12)]
list(zip(range(3), [”fee”, ”fi”, ”fo”, ”fum”
        ]))  # [(0, ’fee’), (1, ’fi’), (2, ’
        fo’)]
# list(zip(range(3), [’fee’, ’fi’, ’fo’, ’fum
        ’], strict=True))  # ValueError: zip
        () argument 2 is longer than argument
         1
[0, *[1, 2, 3], 4, 5, 6]  # [0, 1, 2, 3, 4,
        5, 6]

# A tuple is an immutable ordered collection
        of arbitrary number of elements of
        arbitrary types
a = (
    1,
    ”a”,
    ”a”,
    [0],
)
type(a)  # <class ’tuple’>
a = 1, ”a”, ”a”, [0]  # can also be created
        without the parentheses
a[0]  # 1
a[-1]  # [0]
# a[0] = 0  # TypeError: ’tuple’ object does
        not support item assignment
a[3][0] = 1  # allowed
a = (”a”,)  # a == (’a’,), a tuple. One-item
        tuple requires comma
a = (”a”,)  # a == (’a’,), a tuple
a = ”a”  # a == ’a’, a string
a = (1, 1, 1)
a.count(1)  # 3
a.index(1)  # 0, first occurrence
1 in a  # True
0 not in a  # True

# A set is an immutable unordered collection
        of arbitrary number of elements of
        hashable types with duplicates
        ignored
a = {
    1,
    ”a”,
    ”a”,
}  # t == {1, ’a’}
”a”.__hash__()  # -8760825294024040742, a
        hashable type has __hash__ method
        that returns an int
# a = {[0]} # TypeError: cannot use ’list’ as
         a set element (unhashable type: ’
        list’)
type(a)  # <class ’set’>
type({})  # <class ’dict’>
a = {True, 1}  # t == {True}
a = {1, True}  # t == {1}
a = {False, 0}  # t == {False}
a = {0, False}  # t == {0}
len(a)  # 1
a.clear()  # a == set()
a = {1, 0, 2}  # a == {0, 1, 2}
b = a.copy()  # id(a) != id(b)
a.add(3)  # a == {0, 1, 2, 3}
a.remove(3)  # a == {0, 1, 2}, remove if
        present, KeyError if not
# a.remove(3)  # KeyError: 3
a.discard(2)  # a == {0, 1}, remove if
        present
a.discard(2)  # a == {0, 1}
a.pop()  # 0, a == {1}, Remove and return an
        arbitrary element from the set.
        Raises KeyError if the set is empty.
a.pop()  # 1
# a.pop  # KeyError: ’pop from an empty set’
a = {0, 1, 2, 3}
# arg is any iterable, denoted as b below
a.isdisjoint({0})  # False, a\cap b == \
        varnothing
a.isdisjoint([0])
a.isdisjoint((0,))
a.issubset(a)  # True, a\subseteq b
a <= a  # True, a\subseteq b
a < a  # False, a\subsetneq b
a.issuperset(a)  # True, a\supseteq b
a >= a  # True, a\supseteq b
a > a  # False, a\supsetneq b
a.union({4})  # {0, 1, 2, 3, 4}, a\cup b
a | {4}  # {0, 1, 2, 3, 4}, a\cup b
a.update({1})  # a == {0, 1, 2, 3}, assign a\
        cup b to a
a |= {4}  # a == {0, 1, 2, 3, 4}, assign a\
        cup b to a
a.intersection({3})  # {3}, a\cap b
a & {3}  # {3}, a\cap b
a & {6}  # set(), a\cap b
a.intersection_update(a)  # a == {0, 1, 2, 3,
         4}, assign a\cap b to a
a &= {-1, 0, 1, 2, 3}  # a == {0, 1, 2, 3},
        assign a\cap b to a
a.difference({1})  # {0, 2, 3}, a\setminus b
a - {1}  # {0, 2, 3}, a\setminus b
a.difference_update({1})  # a == {0, 2, 3},
        assign a\setminus b to a
a -= {2}  # a == {0, 3}, assign a\setminus b
        to a
a.symmetric_difference({1, 3})  # {0, 1}, a\
        cup b\setminus(a\cap b)
a ^ {1, 3}  # {0, 1}, a\cup b\setminus(a\cap
        b)
a.symmetric_difference_update({1, 3})  # a ==
         {0, 1}, assign a\cup b\setminus(a\
        cap b) to a
a ^= {1, 3}  # a == {0, 3}, assign a\cup b\
        setminus(a\cap b) to a
{c for c in ”abracadabra” if c not in ”abc”}
         # {’d’, ’r’}, set comprehension

# A dict is a mapping object maps hashable
        values to arbitrary objects
a = dict(one=1, two=2, three=3)
b = {”one”: 1, ”two”: 2, ”three”: 3}
c = dict(zip([”one”, ”two”, ”three”], [1, 2,
        3]))
d = dict([(”two”, 2), (”one”, 1), (”three”,
        3)])
e = dict({”three”: 3, ”one”: 1, ”two”: 2})
f = dict({”one”: 1, ”three”: 3}, two=2)
a == b == c == d == e == f  # True
type(d)  # <class ’dict’>
type({})  # <class ’dict’>
list(d)  # [’two’, ’one’, ’three’], list of
        keys
len(d)  # 3, len(list(d))
d[”two”]  # 2
# d[”a”]  # KeyError: ’a’
d[”four”] = 5  # d == {’two’: 2, ’one’: 1, ’
        three’: 3, ’four’: 5}
d[”four”] = 4  # d == {’two’: 2, ’one’: 1, ’
        three’: 3, ’four’: 4}
del d[”four”]  # d == {’two’: 2, ’one’: 1, ’
        three’: 3}
# del d[”five”]  # KeyError: ’five’
”one” in d  # True
”five” in d  # False
iter(d)  # iter(d.keys())
e.clear()  # e == {}
c = d.copy()
# items(), Return a new view of the
        dictionary’ s items ((key, value)
        pairs).
i = d.items()  # i == dict_items([(’two’, 2),
         (’one’, 1), (’three’, 3)])
list(i)  # [(’two’, 2), (’one’, 1), (’three’,
         3)]
# keys(), Return a new view of the
        dictionary’ s keys.
k = d.keys()  # k == dict_keys([’two’, ’one’,
         ’three’])
list(k)  # [’two’, ’one’, ’three’]
# values(), Return a new view of the
        dictionary’ s values.
v = d.values()  # v == dict_values([2, 1, 3])
list(v)  # [2, 1, 3]
# iter(dictview), Return an iterator over the
         keys, values or items (represented
        as tuples of (key, value)) in the
        dictionary.
# Keys and values are iterated over in
        insertion order since 3.7. This
        allows the creation of (value, key)
        pairs using zip(): pairs = zip(d.
        values(), d.keys()). Another way to
        create the same list is pairs = [(v,
        k) for (k, v) in d.items()].
# x in dictview
# reversed(dictview), reverse iterator
# classmethod fromkeys(iterable, value=None),
         Return a new dictionary with keys
        from iterable and values set to value
        .
d.fromkeys([1, 2, 3])  # {1: None, 2: None,
        3: None}
dict.fromkeys([1, 2, 3])  # {1: None, 2: None
        , 3: None}
# get(key, default=None), Return the value
        for key if key is in the dictionary,
        else default.
d.get(”two”)  # 2
d.get(”four”)  # (None)
# pop(key), pop(key, default), If key is in
        the dictionary, remove it and return
        its value, else return default. If
        default is not given and key is not
        in the dictionary, a KeyError is
        raised.
d.pop(”one”)  # d == {’two’: 2, ’three’: 3}
d.pop(”one”, 10)  # 10
# d.pop(’one’)  # KeyError: ’one’
# popitem(), Remove and return a (key, value)
         pair from the dictionary. Pairs are
        returned in LIFO order since 3.7.
d.popitem()  # (’three’, 3), d == {’two’: 2}
reversed(d)  # reversed(list(d))
# setdefault(key, default=None), If key is in
         the dictionary, return its value. If
         not, insert key with a value of
        default and return default.
d.setdefault(”two”)  # 2, d == {’two’: 2}
d.setdefault(”one”, 1)  # 1, d == {’two’: 2,
        ’one’: 1}
# update(**kwargs), update(mapping, **kwargs
        ), update(iterable, **kwargs), Update
         the dictionary with the key/value
        pairs from mapping or iterable and
        kwargs, overwriting existing keys.
        Return None. update() accepts either
        another object with a keys() method (
        in which case __getitem__() is called
         with every key returned from the
        method) or an iterable of key/value
        pairs (as tuples or other iterables
        of length two). If keyword arguments
        are specified, the dictionary is then
         updated with those key/value pairs:
        d.update(red=1, blue=2).
d.update({”four”: 4}, three=3)  # d == {’two
        ’: 2, ’one’: 1, ’four’: 4, ’three’:
        3}
d.update(two=0)  # d == {’two’: 0, ’one’: 1,
        ’four’: 4, ’three’: 3}
d.update([[”two”, 2]])  # d == {’two’: 2, ’
        one’: 1, ’four’: 4, ’three’: 3}
d.update(([”two”, 2],))
d.update(((”two”, 2),))
d.update([(”two”, 2)])
d | {
    ”one”: 0,
    ”five”: 5,
}  # {’two’: 2, ’one’: 0, ’four’: 4, ’three’:
         3, ’five’: 5}, Create a new
        dictionary with the merged keys and
        values of d and other, which must
        both be dictionaries. The values of
        other take priority when d and other
        share keys.
d |= {
    ”one”: 0,
    ”five”: 5,
}  # d == {’two’: 2, ’one’: 0, ’four’: 4, ’
        three’: 3, ’five’: 5}

class _SNode:
    def __init__(self, data, next=None):
        self.data = data
        self.next = next

class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.head is None

    def __getitem__(self, num):  # O(num)
        if not 0 <= num < self.size:
            raise IndexError(”list index out
                    of range”)
        cur = self.head
        for _ in range(0, num):
            cur = cur.next
        return cur.data

    def __setitem__(self, num, x):  # O(num)
        if not 0 <= num < self.size:
            raise IndexError(”list index out
                    of range”)
        cur = self.head
        for _ in range(0, num):
            cur = cur.next
        cur.data = x

    def push_front(self, x):  # O(1)
        self.head = _SNode(x, self.head)
        if self.tail is None:
            self.tail = self.head
        self.size += 1

    def push_back(self, x):  # O(1)
        node = _SNode(x)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def pop_front(self):  # O(1)
        if self.head is None:
            raise IndexError(”remove from
                    empty list”)
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
            self.tail = None
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError(”peek at empty
                    list”)
        return self.head.data

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError(”peek at empty
                    list”)
        return self.tail.data

    def insert_after_node(self, pos, x):  # O
            (1)
        if self.is_empty():
            raise IndexError(”insert at empty
                     list”)
        if pos is self.tail:
            return self.push_back(x)
        node = _SNode(x, pos.next)
        pos.next = node
        self.size += 1

    def clear(self):  # O(1)
        self.head = None
        self.tail = None
        self.size = 0

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next

class _DNode:
    def __init__(self, data, prev=None, next=
            None):
        self.data = data
        self.prev = prev
        self.next = next

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.size == 0

    def __getitem__(self, num):  # O(min(n,
            size - n)), n = num if num > 0
            else num + size
        if not -self.size <= num < self.size:
            raise IndexError(”list index out
                    of range”)
        if num * 2 >= self.size:
            num -= self.size
        elif num * 2 < -self.size:
            num += self.size
        if num >= 0:
            cur = self.head
            for _ in range(0, num):
                cur = cur.next
        else:
            cur = self.tail
            for _ in range(-1, num, -1):
                cur = cur.prev
        return cur.data

    def __setitem__(self, num, x):  # O(min(n
            , size - n)), n = num if num > 0
            else num + size
        if not -self.size <= num < self.size:
            raise IndexError(”list index out
                    of range”)
        if num * 2 >= self.size:
            num -= self.size
        elif num * 2 < -self.size:
            num += self.size
        if num >= 0:
            cur = self.head
            for _ in range(0, num):
                cur = cur.next
        else:
            cur = self.tail
            for _ in range(-1, num, -1):
                cur = cur.prev
        cur.data = x

    def push_front(self, x):  # O(1)
        node = _DNode(x, None, self.head)
        if self.head is None:
            self.head = self.tail = node
        else:
            self.head.prev = node
            self.head = node
        self.size += 1

    def push_back(self, x):  # O(1)
        node = _DNode(x, self.tail, None)
        if self.tail is None:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size += 1

    def pop_front(self):  # O(1)
        if self.head is None:
            raise IndexError(”pop from empty
                    list”)
        x = self.head.data
        self.head = self.head.next
        self.size -= 1
        if self.head is None:
            self.tail = None
        else:
            self.head.prev = None
        return x

    def pop_back(self):  # O(1)
        if self.tail is None:
            raise IndexError(”pop from empty
                    list”)
        x = self.tail.data
        self.tail = self.tail.prev
        self.size -= 1
        if self.tail is None:
            self.head = None
        else:
            self.tail.next = None
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError(”peek at empty
                    list”)
        return self.head.data

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError(”peek at empty
                    list”)
        return self.tail.data

    def insert_after_node(self, pos, x):  # O
            (1)
        if self.is_empty():
            raise IndexError(”insert at empty
                     list”)
        if pos is self.tail:
            return self.push_back(x)
        node = _DNode(x, pos, pos.next)
        pos.next.prev = node
        pos.next = node
        self.size += 1

    def insert_before_node(self, pos, x):  #
            O(1)
        if self.is_empty():
            raise IndexError(”insert at empty
                     list”)
        if pos is self.head:
            return self.push_front(x)
        node = _DNode(x, pos.prev, pos)
        pos.prev.next = node
        pos.prev = node
        self.size += 1

    def remove_node(self, node):  # O(1)
        if self.head is None:
            raise IndexError(”remove from
                    empty list”)
        if self.head is node:
            return self.pop_front()
        elif self.tail is node:
            return self.pop_back()
        node.prev.next = node.next
        node.next.prev = node.prev
        self.size -= 1
        return node.data

    def clear(self):  # O(1)
        self.head = None
        self.tail = None
        self.size = 0

    def __iter__(self):
        cur = self.head
        while cur is not None:
            yield cur.data
            cur = cur.next

    def __reversed__(self):
        cur = self.tail
        while cur is not None:
            yield cur.data
            cur = cur.prev

class BlockListDeque:  # Both ends IO
        amortized O(1)
    def __init__(self, block_size=64,
            front_step=1, back_step=1):
        if block_size <= 0:
            raise ValueError(”deque
                    block_size must >0”)
        if front_step <= 0:
            raise ValueError(”deque
                    front_step must >0”)
        if back_step <= 0:
            raise ValueError(”deque back_step
                     must >0”)
        self.block_size = block_size
        self.front_step = front_step
        self.back_step = back_step
        self.data = [[None] * self.block_size
                 for _ in range(self.
                front_step + self.back_step -
                 1)]
        self.block_begin = self.front_step -
                1
        self.block_end = self.block_begin + 1
        self.in_begin = (self.block_size - 1)
                 // 2
        self.in_end = self.in_begin
        self.size = 0

    def __len__(self):  # O(1)
        return self.size

    def is_empty(self):  # O(1)
        return self.size == 0

    def capacity(self):  # O(1)
        return self.block_size * len(self.
                data)

    def is_valid(self):  # O(1)
        return self.size == self.block_size *
                 (self.block_end - self.
                block_begin - 1) + self.
                in_end - self.in_begin

    def __getitem__(self, index):  # O(1)
        if not -self.size <= index < self.
                size:
            raise IndexError(”deque index out
                     of range”)
        if index < 0:
            index += self.size
        block_offset, in_block = divmod(
            self.in_begin + index,
            self.block_size,
        )
        return self.data[self.block_begin +
                block_offset][in_block]

    def __setitem__(self, index, x):  # O(1)
        if not -self.size <= index < self.
                size:
            raise IndexError(”deque index out
                     of range”)
        if index < 0:
            index += self.size
        block_offset, in_block = divmod(
            self.in_begin + index,
            self.block_size,
        )
        self.data[self.block_begin +
                block_offset][in_block] = x

    def reserve_front(self, blocks=None):  #
            O(1)
        if blocks is None:
            blocks = self.front_step
        if blocks <= self.block_begin:
            return
        new = [[None] * self.block_size for _
                 in range(blocks)]
        new.extend(self.data)
        self.data = new
        self.block_begin += blocks
        self.block_end += blocks

    def reserve_back(self, blocks=None):  # O
            (1)
        if blocks is None:
            blocks = self.back_step
        if self.block_end + blocks <= len(
                self.data):
            return
        self.data.extend([[None] * self.
                block_size for _ in range(
                blocks)])

    def shrink_front_to_fit(self):  # O(1)
        new = self.data[self.block_begin :]
        self.data = new
        self.block_end -= self.block_begin
        self.block_begin = 0

    def shrink_back_to_fit(self):  #
            amortized O(1)
        for _ in range(self.block_end, len(
                self.data)):
            self.data.pop()

    def push_front(self, x):  # amortized O
            (1)
        if self.in_begin != 0:
            self.in_begin -= 1
        else:
            if self.block_begin == 0:
                self.reserve_front()
            self.block_begin -= 1
            self.in_begin = self.block_size -
                 1
        self.data[self.block_begin][self.
                in_begin] = x
        self.size += 1

    def push_back(self, x):  # amortized O(1)
        if self.in_end != self.block_size:
            self.in_end += 1
        else:
            if self.block_end == len(self.
                    data):
                self.reserve_back()
            self.block_end += 1
            self.in_end = 1
        self.data[self.block_end - 1][self.
                in_end - 1] = x
        self.size += 1

    def pop_front(self):  # O(1)
        if self.is_empty():
            raise IndexError(”pop from empty
                    deque”)
        x = self.data[self.block_begin][self.
                in_begin]
        self.data[self.block_begin][self.
                in_begin] = None
        self.size -= 1
        if self.in_begin != self.block_size -
                 1:
            self.in_begin += 1
        else:
            self.block_begin += 1
            self.in_begin = 0
        return x

    def pop_back(self):  # O(1)
        if self.is_empty():
            raise IndexError(”pop from empty
                    deque”)
        x = self.data[self.block_end - 1][
                self.in_end - 1]
        self.data[self.block_end - 1][self.
                in_end - 1] = None
        self.size -= 1
        if self.in_end != 1:
            self.in_end -= 1
        else:
            self.block_end -= 1
            self.in_end = self.block_size
        return x

    def peek_front(self):  # O(1)
        if self.is_empty():
            raise IndexError(”peek from empty
                     deque”)
        return self.data[self.block_begin][
                self.in_begin]

    def peek_back(self):  # O(1)
        if self.is_empty():
            raise IndexError(”peek from empty
                     deque”)
        return self.data[self.block_end - 1][
                self.in_end - 1]

    def clear(self):  # O(len(self.data))
        num_blocks = len(self.data)
        self.data = [[None] * self.block_size
                 for _ in range(num_blocks)]
        self.block_begin = self.front_step -
                1
        self.block_end = self.block_begin + 1
        self.in_begin = (self.block_size - 1)
                 // 2
        self.in_end = self.in_begin
        self.size = 0

    def __iter__(self):
        block = self.block_begin
        index = self.in_begin
        for _ in range(self.size):
            yield self.data[block][index]
            index += 1
            if index == self.block_size:
                index = 0
                block += 1

    def __reversed__(self):
        block = self.block_end - 1
        index = self.in_end - 1
        for _ in range(self.size):
            yield self.data[block][index]
            index -= 1
            if index < 0:
                index = self.block_size - 1
                block -= 1
```